

Using ROS2 to Build an Integrated SLAM System in Rust

Gabriel J. Ferrer¹

Abstract—Building a SLAM-based robot control system in Rust requires building a series of ROS2 nodes that concurrently acquire sensory information, update the map, make control decisions based on the map, but also make control decisions based on regular sensor inputs. This also requires making careful use of Rust’s concurrency constructs. We describe lessons learned from a successful implementation of a SLAM-based control system using Rust atop ROS2, making use of the `r2r` and `smol` crates.

I. INTRODUCTION

As part of a research project developing a SLAM algorithm, we made the design decision to implement the algorithm in Rust. In this paper, we describe the system architecture, how we realized the system architecture in Rust via ROS2 [5], complications that ensued from handling concurrency, and building additional Rust utilities for visualization. We discuss the resulting system throughput as well.

II. SYSTEM ARCHITECTURE

We have been developing a trajectory-based SLAM algorithm, BitSLAM, employing particle filters [1] [2] [3]. In trajectory-based SLAM, the map is built by designating all areas the robot visits as free space and all collisions as obstacles. We implemented our system aboard an iRobot Create3 robot [4] connected to a Raspberry Pi.

Our system consists of two ROS2 nodes that run indefinitely: `BitSlamNode` and `OnPathBumpIrNode`. `OnPathBumpIrNode` communicates with the iRobot Create3 by subscribing to `odom`, `ir_intensity`, and `hazard_detection` and publishing to `cmd_vel_stamped`, and with `BitSlamNode` by publishing to `bit slam_obstacles` and subscribing to `bit slam_on_path`. `BitSlamNode` also subscribes to `odom`.

It also includes a couple of message-sending nodes that send their message and quit. `StopNode` publishes to `bit slam_explorer_stop`, to which `OnPathBumpIrNode` subscribes, stopping the robot when it receives a message. `SaveNode` publishes to `bit slam_save`, to which `BitSlamNode` subscribes, saving the current map when it receives a message.

The algorithm of `OnPathBumpIrNode` is as follows:

- The node is always in one of three states: Forward, Turning, or Stopped.
- Receiving a hazard message or an IR message with an IR reading above a preset threshold and not in the Stopped state results in the following:

```
pub struct NodeSpec {
    node: Arc<Mutex<Node>>,
    futures: Vec<Pin<Box<dyn Future<Output
        = ()> + Send + 'static>>>,
    period: u64,
}
```

Fig. 1. NodeSpec definition

- The state changes to Turning.
- The node publishes an obstacle message to `bit slam_obstacles`.
- Receiving an odometry message results in the following:
 - The node publishes a pose message to `bit slam_obstacles`.
 - The node publishes a `TwistStamped` object corresponding to its current state to `cmd_vel_stamped`.
- Receiving an `on_path` message changes the state to Forward only if it is currently Turning.

`BitSlamNode` interacts with `OnPathBumpIrNode` as follows:

- When it receives a message on `bit slam_obstacles`, it updates its map.
- When it receives a message from `odom`, it employs an all-pairs-shortest-paths algorithm to find a shortest path through its grid map to every unvisited square. If the robot’s heading, derived from `odom`, is consistent with one of those paths, it publishes an `on_path` message.

III. AN EVENT-DRIVEN FOUNDATION

We employed the `r2r` crate [10] to implement `OnPathBumpIrNode` and `BitSlamNode` in Rust. ROS2 nodes are created in `r2r` using the `r2r::Node` data type. The subscribe method of the `Node` data type returns a `Stream` of objects of the data type associated with the topic to which it is subscribed. A natural implementation of ROS2 subscriptions is an event-driven model, where each arriving message is treated as an event to be handled. We built infrastructure to enable this approach. To facilitate this implementation, we used the `smol` crate [9] for implementing `async/await`.

Our foundational data type is `NodeSpec`; definition is shown in Figure 1. `NodeSpec` objects contain an `Arc<Mutex<Node>>`, a `Vec` of futures, and a cycle period. The initialization of a `NodeSpec` object involves creating an `r2r::Context`, initializing an `r2r::Node` from that `r2r::Context`, and then wrapping the node in

¹Gabriel Ferrer, Professor of Computer Science, Hendrix College, Conway, AR 72032, USA ferrer@hendrix.edu

```

pub fn subscribe<
  P: WrappedTypesupport + Send + 'static,
  F: FnMut(P, Arc<Mutex<Node>>) + Send + 'static,
>(
  &mut self, topic: &str, handler: F,
) -> anyhow::Result<()> {
  let mut subscriber = {
    let mut node = smol::block_on(self.node.lock());
    node.subscribe::<P>(topic, QosProfile::sensor_data())
      .map_err(|e| anyhow::anyhow!("Subscribe failed: {}", e))?
  };

  let node = self.node.clone();
  self.futures.push(Box::pin(async move {
    let mut handler = handler;
    while let Some(msg) = subscriber.next().await {
      handler(msg, node.clone());
    }
  }));
  Ok(())
}

```

Fig. 2. Setting up subscriptions

```

pub fn run(self, running: Arc<AtomicCell<bool>>) -> anyhow::Result<()> {
  smol::block_on(async {
    for future in self.futures {
      smol::spawn(future).detach();
    }
    while running.load() {
      let timeout = std::time::Duration::from_millis(self.period);
      {
        let mut node = smol::block_on(self.node.lock());
        node.spin_once(timeout);
      }
      smol::Timer::after(timeout).await;
    }
  });
  Ok(())
}

```

Fig. 3. Spinning the node

an Arc and Mutex. Creating a publisher is a straightforward wrapping of the `Node::create_publisher` method, returning an `r2r::Publisher` object.

Much more involved is setting up a subscription. It is important to have a clear conceptual model of the connection between creating the subscription and executing the node, because `Node::subscribe` returns a `Stream` of ongoing messages. Our approach, shown in Figure 2, is to wrap each `Stream` in a future that contains a loop that grabs each arriving message and passes it to a function given as a parameter. This future is stored in the object's repository of futures, which in turn are spawned in the `run` method (see Figure 3). After all the futures are spawned, we start a loop

to spin the node until a flag variable's value is set to `false`.

The means by which this enables an event-driven development model is that we can pass a closure every time we use the `subscribe` method to create a subscription. Those closures, in turn, get activated every time a message arrives.

IV. MORE INFRASTRUCTURE

We also sought flexibility in our node implementations. We wanted it to be possible, when implementing a node, to run it as an independent executable or to run it as part of a network of nodes in an executable. We also wanted to be able to standardize the command-line arguments to control the node's parameters in a standardized way, as well as have some standardized means of documenting those

```
fn main() -> anyhow::Result<()> {
    let nodes: Vec<Box<dyn RunnableNode>> =
    vec![
        Box::new(BitSlamNode::default()),
        Box::new(OnPathBumpIrNode::default()),
    ];
    run_nodes(nodes)
}
```

Fig. 4. Creating and Running RunnableNode objects

```
let status = Arc::new(Mutex::new
    (OnPathStatus::new(
        args.get_value("--max-ir")?,
        spec.publisher::<Ros2String>
            (&pubs[1])?,
    )));

let hazard_status = status.clone();
spec.subscribe(&subs[0], move
    |hazards: HazardDetectionVector, node|
    {
        if let Some(mut status) =
            hazard_status.try_lock() {
            if let Err(e) =
                status.handle_hazards(node,
                    &hazards) {
                eprintln!("Error: {e}");
            }
        }
    }
    )?;
```

Fig. 5. Hazard Subscription

parameters as well as the topics to which it publishes and subscribes. To these ends, we created the `RunnableNode` trait. In Figure 4, we show the interaction between this trait and the `run_nodes` function. That function invokes the `RunnableNode::spec` method to generate a `NodeSpec` object for that implementation of the trait. It then invokes the `run` method for each generated `NodeSpec` object, allowing all of them to run concurrently.

V. IMPLEMENTING A PROJECT NODE

Implementing `OnPathBumpIrNode` begins with creating a data type that implements the `RunnableNode` trait. Setting up subscriptions as event handlers passing around closures requires any shared state to be locked. To that end, we created a data type, `OnPathStatus`, to represent the shared state. To try to reduce the complexity of the event handlers, we moved as much of their implementation as possible into methods of this data type.

We see this in Figure 5, the subscription to the `hazard_detection` topic. We start by creating a shared `OnPathStatus` object, which we then clone for each closure. We decided to use `try_lock` in order to avoid the possibility of deadlocks within the node. When a hazard

Path	Particles	Max Iteration (ms)
A	1000	2.35
A	2000	7.57
B	1000	3.82
B	2000	6.89

TABLE I
MAXIMUM PARTICLE-FILTER ITERATION (MS)

is detected, it changes its status to indicate that it needs to be turning. The program is designed so that it only publishes motor commands in response to `odom` messages, so the turn itself happens later, as shown in Figure 6.

The odometry subscription handler is divided into two parts. First, it handles status updates based on the odometry value received. Second, it asks the status to create a `TwistStamped` object to be published to the motors. Since odometry messages are received at very regular intervals, other sensor values are incorporated into the status, to be queried whenever odometry arrives.

VI. SENDING, SAVING, AND VISUALIZING MAPS

On each iteration of the particle filter, `BitSlamNode` publishes the map associated with the current best particle. For the `bitslam_occupancy_grid` topic, it publishes the map in the best particle as an `OccupancyGrid` object. For the `bitslam_maps` topic, it employs the `serde` [6] and `serde_json` [7] crates to serialize the entire particle data structure, including the complete map, into JSON strings, publishing them using the `ROS2 String` type.

We were specifically interested in benchmarking BitSLAM's performance using the SLAM toolbox. Publishing maps as `OccupancyGrid` objects facilitates this. Because `OccupancyGrid` objects cannot represent all aspects of a BitSLAM map, it is advantageous to serialize into JSON to enable the complete data structure to be published and saved.

When a `BitSLAMNode` receives a `save` message on the `bitslam_save` topic, it serializes all of the particles in the particle filter into a single combined JSON string and saves them into a timestamped file.

By saving all of the particles in the filter, we can deserialize and visualize them offline. We employed the `egui` crate [8] to build a graphical visualizer that takes the serialized JSON of all of the particles in the particle filter and displays them. The visualizer, depicted in Figure 7, allows the user to navigate among all of the particles in the filter. The user can scroll through them in order of quality, to see the best-quality map according to its internal quality metric, or to see the map that is closest to the externally measured ground truth. The combination of readily available and easily employed JSON serialization and a straightforward GUI crate made creating the visualizer very straightforward.

VII. THROUGHPUT

The Rust implementation of our particle filter performed very well. To demonstrate this, we had our robot travel a path to map each of two rooms. **Path A** results from running our robot with a simple obstacle-avoiding algorithm in an 3x4

```

let odom_status = status.clone();
let motor_publisher = spec.publisher::<TwistStamped>(&pubs[0])?;
spec.subscribe(&subs[2], move |odom: Odometry, node| {
    if let Some(mut status) = odom_status.try_lock() {
        if let Err(e) = status.handle_odom(&odom, node.clone()) {
            eprintln!("Error {e} when handling odometry");
        }
        match status.state.twist_stamped(node) {
            Ok(twist) => {
                if let Err(e) = motor_publisher.publish(&twist) {
                    eprintln!("Couldn't publish twist: {e}");
                }
            }
            Err(e) => {
                eprintln!("Couldn't build twist: {e}")
            }
        }
    }
})?;

```

Fig. 6. Odometry Subscription

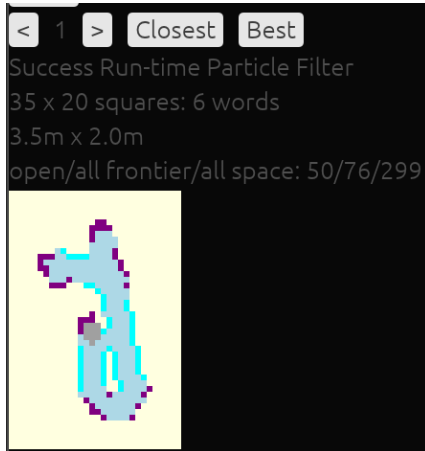


Fig. 7. Visualization

meter empty room. **Path B** results from using BitSLAM to map a second room of similar size containing a couple of tables, one along a wall and another in the center.

In Table I, we show the maximum iteration time in milliseconds for the particle filter when mapping each of these paths. Unsurprisingly, doubling the number of particles roughly doubles the iteration time. As our iteration times remain safely under 10 milliseconds, and with 2000 particles producing reasonably satisfactory maps, the Rust implementation achieved our throughput goals.

VIII. CONCLUSION

The system works as intended. Our robot runs indefinitely while building a map, and using Rust was a key part of our success. Being able to rely upon the compiler to avoid data races and being able to configure `smol` to avoid deadlocks enabled us to meet the promise of fearless concurrency. The

`r2r` crate was sufficient for building a reasonably intuitive event-driven framework for our ROS2 nodes.

That said, upon completing the project we were left with the impression that the complexity of the resulting code was greater than the complexity of the application logic. We found ourselves writing significantly more boilerplate code than we would have liked, and we created numerous data structures to try to abstract away numerous details. More study, and feedback from the community, may be helpful in determining whether we are using the right abstractions, or if they truly correspond to the underlying problem complexity.

REFERENCES

- [1] A. I. Eliezer and R. Parr, "Hierarchical Linear/Constant Time SLAM Using Particle Filters for Dense Maps," 2005 Conference on Neural Information Processing Systems.
- [2] M. Montemerlo and S. Thrun, "Simultaneous localization and mapping with unknown data association using FastSLAM," IEEE ICRA, Taipei, Taiwan, 2003, pp. 1985-1991.
- [3] Y. Chen et al., "Trajectory-Based SLAM for Indoor Mobile Robots with Limited Sensing Capabilities," 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 2023, pp. 9147-9154, doi: 10.1109/IROS55552.2023.10341518.
- [4] https://iroboteducation.github.io/create3_docs/api/ros2/. iRobot Create3 ROS2 API. Retrieved March 2, 2026.
- [5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics vol. 7, May 2022.
- [6] David Tolnay, "Serde" <https://crates.io/crates/serde>. Retrieved September 22, 2026.
- [7] David Tolnay, https://crates.io/crates/serde_json. "Serde JSON" Retrieved September 22, 2026.
- [8] Emil Ernerfeldt, "egui" <https://crates.io/crates/egui>. Retrieved September 22, 2026.
- [9] Taiki Endo, "SMOL" <https://crates.io/crates/smol>. Retrieved August 14, 2026.
- [10] Martin Dahl, "r2r" <https://crates.io/crates/r2r>. Retrieved August 14, 2026.
- [11] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world", Journal of Open-Source Software 6(61), 2021, https://github.com/SteveMacenski/slam_toolbox. Retrieved August 10, 2026.